Class 5 -Basic Specification Writing for Agentic Use

v2 by Dennis Tam

What you will learn

- What is a basic specification
- What are the key components of a minimally functional specification
- What can be prepared by non-technical audience
- What can be parepared by ‘AI’

General Flow of a vibe-coding approach

1. Define your problem
2. Explore your problem with help of LLM(sometimes we don’t know what we don’t know)
3. Define & explore your solution with help of LLM
    a. Use interrogative prompt to help degine a project strcuture
4. Once approving the plan, ask AI to build according to the plan (ie. build.md)

Why this is important

- LLM, no matter how advanced, is an auto-complete
- Specification provides the framework for ‘AI’ to auto-complete around
- A muddy input will generate a muddy output

Models has got good...why is this needed?

- Model intelligence can be proxied by how well it can auto complete an acceptable solution given minimal instruction
  - More ‘intelligence’ == More cost
  - More ‘intelligence’ == Less first-time mistakes
  - More ‘intellgence’ == More self-aware to persistently fix bugs
  - More ‘intellgence’ == More self-aware to spot gaps at ‘planning phase’
  - More ‘intellgence’ == Generally better ‘taste’ and ‘details’

Models has got good...why is this needed?

- Model intelligence doesn’t remove the need for a specification
  - More intelligence != Knows what you are thinking
  - More intelligence != Knows what your business context are
  - More intelligence != Knows what your business logic are

Before implementation

- Do not try to ‘invent problems’
- Have a clear logic of the problem you are trying to solve
  - ie. Different roles (client...admin...)
  - ie. Different flows (client request...admin accept...)
- Identify business needs
  - ie. Admin has to have a friendly portal to overview
- Try to fill in gaps in your business logic
  - ie. Multiple clients...one administrator

Block 1 - Intent

- The first line usually is for agent to identify the purpose of the document
- The agent has to have a rough overview of the project goal to align further actions

Build Guide: Customer Management System

This is the canonical specification for building, verifying, and deploying a self-contained customer management system. It is written for a coding agent implementing the project end to end. The agent should implement the product from this document, run the verification checks, and report evidence. It must never place secrets in source files, Git, chat, or Markdown.

1. Product Goal

The business offers appointment-based services to its customers (for example, pet boarding stays, lessons, consultations, or service visits — the booking model below fits any bookable service with a start and end). Customers currently enquire by phone or message, and staff track availability by hand. This system replaces that with a single web application that has two faces:

- **Client portal** — customers register, manage their account, create booking enquiries, receive status updates, and message the business.
- **Admin portal** — the business reviews booking enquiries, accepts them, or requests a reschedule, chats with customers, and sees bookings on a calendar with summary statistics.

The system is self-contained: bookings are created by clients inside the application. There is no external form, spreadsheet, or third-party intake feed.

The finished system has one local development setup and one production deployment. The production deployment serves both the frontend and the API from a single project.

Block 2 - Roles

- Explain the access control and role of the intended use
  - ie - Who will use the system and how

2. Roles and Access

Role	Count	Access
Client	Many	Client portal only: own profile, own bookings, own messages
Admin	Exactly one account	Admin portal only: all bookings, all clients, calendar, statistics, all conversations

Authentication rules:

- Clients self-register with email and password. Passwords are hashed (bcrypt/argon2) and never stored or logged in plain text.
- The admin account is created once through a protected one-time setup route (or seeded directly in the database) and is not self-service. There is no public registration for the admin role.
- A client who is not logged in can only reach the login/registration pages.
- A client can never read or modify another client's bookings, profile, or messages. Every client-scoped query is filtered by the authenticated user's ID.
- The admin account cannot create bookings on behalf of clients in v1. Its write actions are: accept, request reschedule, reply to messages.

Block 3 - Functional Requirement

3. Functional Requirements

3.1 Client Portal

Authentication & account

- Register: name, email (unique), phone (optional), password, confirm password.
- Login and logout with session handling (server-side sessions or signed JWT).
- Account management: view and edit name, phone, and password. Email is the login identity and is not editable in v1.
- A client sees only their own data everywhere.

- Explains the functions of the application given different perspective of users

Bookings

- Create a booking enquiry with: service type (from a configurable list), start date, end date (same-day bookings allowed; end must not precede start), notes (optional free text).
- A new booking starts with status `pending`.
- List own bookings with status badges and timestamps of the latest status change.
- A booking may be withdrawn by the client while it is `pending` (status becomes `cancelled`). Accepted bookings cannot be cancelled by the client in v1.
- Cancelled bookings remain visible in the client's history as `cancelled`.

Updates

- When the admin accepts, requests a reschedule, or the client withdraws a booking, the booking's status and `updated_at` change. The client sees this immediately in their booking list.
- Each status change also appends an entry to a notifications list for that client ("Your booking for 12–15 Oct was accepted."). The client portal has a simple notifications view showing unread/read state; opening it marks entries as read.

Messaging

- A client has one ongoing conversation with the admin (one thread per client, not per booking, in v1).
- The client can send a text message at any time; messages appear in order with sender attribution and timestamps.
- The client is told when the admin has replied, either via the notifications feed or an unread indicator on the message thread.

3.2 Admin Portal

Authentication

- A single login screen for the admin account. There is no admin registration page.

Booking review

- A dashboard lists all booking enquiries, newest first, with client name, service, dates, and status.
- Filter by status (`pending`, `accepted`, `reschedule_requested`, `cancelled`) and search by client name or email.
- From a pending booking, the admin can:
  - **Accept** — status becomes `accepted`; the client is notified.
  - **Request reschedule** — the admin enters a proposed date range and a short reason; status becomes `reschedule_requested`; the client is notified and can respond in the c
- Accepted bookings are visible in the calendar and statistics. `reschedule_requested` and `cancelled` bookings are not.

Chat

- A conversation list showing every client who has a message thread, with unread counts.
- An open thread shows the full history with sender attribution and timestamps, and a reply box.
- Replies from the admin are delivered to the client's thread and notifications.

Calendar overview

- A month-view calendar occupying the main panel of the admin home screen.
- Each day cell shows count badges or chips for accepted bookings that cover that date (inclusive of start and end dates).
- Clicking a day lists the accepted bookings on that day with client and service details.
- A control switches the calendar between "count chips" and a "spaced occupancy" display; either is acceptable as long as overlapping multi-day bookings are clearly represented

Statistics panel

- Total bookings overall, split by status.
- Bookings this month and this week.

Human + AI Collaboration

- Block 1-3 can be clearly defined by human (Business stuff)
- Block 4-8 can be extended by machine (Technical stuff)

Block 4 - Data model

- Describe database table and its relationship

4. Data Model

The database is **Neon** (serverless PostgreSQL). The schema below is the minimum; additional indexes or columns for convenience are allowed but existing columns must not be renamed or retyped.

```
CREATE TABLE users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    name          TEXT NOT NULL,
    phone         TEXT NOT NULL DEFAULT '',
    role          TEXT NOT NULL DEFAULT 'client'
                  CHECK (role IN ('client', 'admin')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE bookings (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id   UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    service     TEXT NOT NULL,
    start_date  DATE NOT NULL,
```

Block 5 - API model

- Describe how request and data is routed within different component (aka. backend)

5. API Specification

All routes are JSON. Errors return a consistent shape: `{"error": "<machine-readable code>", "message": "<human-readable text>"}` with an appropriate HTTP status.

5.1 Auth

Method & Path	Who	Description
`POST /api/auth/register`	Public	Create a client account; returns the created session
`POST /api/auth/login`	Public	Log in; returns a session for client or admin
`POST /api/auth/logout`	Authenticated	End the current session
`GET /api/auth/me`	Authenticated	Return the current user's profile and role

5.2 Client Account

Method & Path	Who	Description
`PATCH /api/me`	Client	Update name, phone, or password

Block 6 & 7 - Architecutre and Environement

- What’s the tech stack? How is it pieced together?
- How to store sentitive data

6. Architecture

```
React/Vite frontend (two portals, one build)
    -> FastAPI application
    -> Neon PostgreSQL (serverless)
    -> Session/auth layer backed by the database
```

- **Frontend**: one React/Vite application that renders either the client portal or the admin portal based on the authenticated user's role after login. Two top-level route trees: `/app/...` for clients and `/admin/...` for the admin. Unauthenticated visitors are redirected to `/login`. The production build must call the API by same-origin relative paths only — never `localhost`.
- **Backend**: FastAPI, structured as `main.py` (app factory), `routers/` (one module per section of the API table), `db.py` (Neon connection pool), `auth.py` (hashing, session issue/verify, role guard), `models.py` (row mappings), `schemas.py` (request/response validation). Route handlers enforce the role column on every request; role is never taken from the client.
- **Database**: Neon PostgreSQL. A single `DATABASE_URL` (pooled connection string) is the only database configuration. Schema changes are applied through a migration step run by the agent; the user is not asked to paste SQL by hand.
- **Deployment**: one hosting project serving the built frontend as static files and the API under `/api/*`. Server-side environment variables (`DATABASE_URL`, `SESSION_SECRET`) are set in the hosting dashboard, never in source.

7. Environment and Secrets

Local development uses a root `.env` file (git-ignored):

```
DATABASE_URL=<Neon pooled connection string>
SESSION_SECRET=<random secret for signing sessions>
VITE_API_URL=http://localhost:8000   # local dev override only; never used in production
```

Rules:

Block 8 - Execution Phases

- How the agent will complete the task

8. Build Phases

Follow these phases in order. Do not skip a checkpoint.

Phase 1: Project setup

Create the repository structure: `src/backend/`, `src/frontend/`, root `.env.example`, `.gitignore`, and a `README.md` stub. Confirm the `.gitignore` covers all environment files. Record non-secret project r
hosting project name) in `README.md`.

Checkpoint: the folder structure exists and no secret value is tracked anywhere.

Phase 2: Database provisioning

Create (or select) the Neon project for this application. Record the project name as metadata. Apply the schema from Section 4 through a migration step run by the agent, then verify every table exists by inspe

Checkpoint: all four tables and their indexes exist in the intended Neon project.

Phase 3: Backend

Implement in slices, running the API after each slice:

1. Configuration loading, Neon connection pool, and `/api/health`.
2. Auth: register, login, logout, `me`, session verification, role guard.
3. Client account endpoints and booking CRUD + status transitions.
4. Messaging and notifications.
5. Admin endpoints: booking review, conversations, overview (calendar + stats).

Checkpoint: every route in Section 5 responds correctly, including the rejection cases (wrong role, other client's resource, invalid date range)

Block 9 & 10 - Guardrails, Error handling

- Key pointers of task safety
- Edge cases of unexpected user behaviour

9. Guardrails

- Never commit or print `.env`, `DATABASE_URL`, `SESSION_SECRET`, session tokens, or password hashes in plain text anywher
- The `role` column is the only source of authorization truth; it is always read server-side from the session, never from reques
- Every client-scoped query is filtered by the authenticated user ID.
- Booking dates are inclusive on both ends.
- Passwords are hashed with bcrypt or argon2; the application never logs authentication payloads.
- Keep the backend in FastAPI and the frontend in React/Vite.
- One database (Neon), one backend, one frontend build, one production project.
- Status transitions are the only allowed mutations on `bookings.status`; handlers reject transitions that are not in Section 5

10. Error Handling and Edge Cases

- Duplicate registration email → `409` with a clear message.
- Login with wrong credentials → generic `401` that does not reveal whether the email exists.
- `end_date` before `start_date` → `400` at creation and at reschedule proposal.
- Client withdraw on a non-`pending` booking → `409`.
- Admin accept/reschedule on a non-`pending` booking → `409`.
- Missing or expired session → `401`; wrong role → `403`.
- Database unavailable → `503` from `/api/health` with a plain-language message.
- Empty states (no bookings, no messages, no notifications) render as friendly empty panels, not blank screens or errors.

Block 11 - Project Completion

- How shall the project be marked as completed?

11. Definition of Done

A build is complete when:

1. `/api/health` succeeds locally and in production.
2. A client can register, log in, edit their account, and log out.
3. A client can create a booking and see it as `pending`.
4. The admin sees the enquiry, accepts it, and the client's list and notifications update.
5. The admin can request a reschedule with proposed dates and a reason, and the client sees it.
6. Client and admin can exchange messages, with correct unread behavior.
7. The admin calendar shows accepted bookings on all inclusive dates, and the statistics panel matches the underlying data.
8. Cross-client access attempts (reading another user's booking, thread, or notifications) are rejected.
9. No secret value appears in the repository, logs, or this document.
10. Production serves the frontend and API from one project with the full lifecycle verified in the browser.

What was discussed?

- Basic component of a technical specification
- For a complete, robust documentation
  - Refer to Complete_documentation_template.docx
- Next question: How do we write build.md
  - Utilise seedprompt_build.md

Refer to seedprompt_build.md

- Purpose:
  - Help interview user to brainstorm the business logic and requirement of the project
- Format:
  - Interrogative interviewing
    - LLM will ask question turn by turn to help user
  - Follow the question turn by turn
  - Approve the business logic to enable the AI to keep building for you

Refer to seedprompt_build.md

- How:
  - Upload the seedprompt_build.md to an LLM
    - Can be Gemini/Grok/Github-copilot
- Purpose:
  - Help interview user to brainstorm the business logic and requirement of the project
- Format:
  - Interrogative interviewing
    - LLM will ask question turn by turn to help user
  - Follow the question turn by turn
  - Approve the business logic to enable the AI to keep building for you